Competitive Programming

The Brief

Explainer for Salman Adnan's portfolio

In one paragraph

361 C++ solutions Salman wrote while training for programming contests (Codeforces, CSES, AtCoder, CS Academy, and the CS 334 course at Habib University), filed by topic. Three Python tools make the collection checkable: one compiles every file, one stress-tests the eight data-structure solutions against brute force, and one tests every solution that was fixed after a line-by-line review. Thirty files he studied but did not write are kept apart and never counted. A glossary is at the end.

Contents

1 The problem the repository solves **1**

2 What is in it **2**

3 How a solution is written **2**

4 The core: range queries **2**

5 Verified, not assumed **3**

6 Code that is not Salman's **3**

7 Honest limits **4**

8 The story **4**

9 Glossary **4**

1 The problem the repository solves

Competitive programming is solving short algorithm puzzles against a clock, with an automated judge running your program on hidden inputs. Salman did it full time for about eight months, by his own account. The work sat in one untracked folder: duplicates, empty placeholders, editor backups, half-finished attempts, and reference code from other people mixed in with his own. The repository turns that folder into something a reader can browse and a machine can check, and is careful to claim only what it can re-check.

2 What is in it

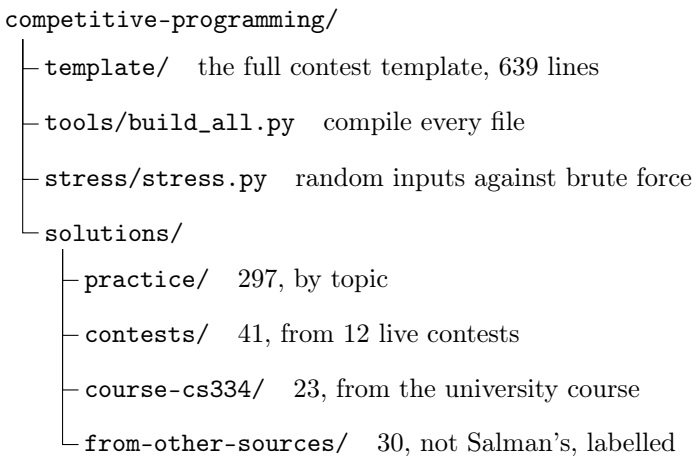


Figure 1: The repository at a glance (398 tracked files in all).

The practice folders are graphs (65), dynamic programming (36), number theory (48), a large mixed set (101), and smaller sets on range queries, bitmasks, prefix sums, two pointers, CS Academy and AtCoder. Folder names say what a training set was meant to practise, not always what each answer needed: 15 of the graphs files build no graph at all, and one of them is a sparse table.

3 How a solution is written

A contest program reads from standard input and writes to standard output, nothing else. Most of Salman's files (267 of 361) paste a shared *template* at the top, then put the answer in a `solve()` function under a `SOLVE` banner. Reading any file means searching for that banner.



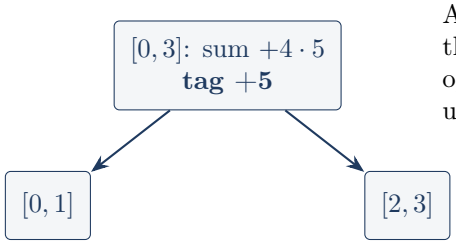
Figure 2: Template use across Salman's 361 files. The template shrank over time.

The full template (the copy in `template/`) is a library of number theory, a sieve, DSU, a lazy segment tree, a Fenwick tree, graph algorithms, matrix power and fast I/O. File dates show it came first and was then cut down to a trimmed version keeping only type aliases, loop macros, gcd, the `re`/`ps` I/O helpers and debug macros. Local-only code (timing, reading from a file) is behind a `LOCAL == SALMAN` preprocessor guard that is off on every judge.

4 The core: range queries

The eight CSES Range Queries solutions are the technical heart. Each problem has up to 200,000 numbers and 200,000 questions, so a loop per question is far too slow; each structure answers in $O(\log n)$ or $O(1)$.

Problem	Idea
Range Xor, Forest Queries	prefix tables (1D XOR, 2D sums with inclusion-exclusion)
Dynamic Range Sum	Fenwick tree; "set" becomes "add the difference"
Dynamic Range Minimum	segment tree with point updates
Hotel Queries	max segment tree that walks down to the leftmost fit
Range Update Queries	lazy segment tree, range add
Range Updates and Sums	lazy tree with two tags, add and assign, folded so one is pending
Polynomial Queries	lazy tag that is a line $c+d\,i$, so progressions combine by addition



A covered node updates its own sum and leaves a tag; the tag is pushed to the children only when a later operation needs to look inside. That keeps a range update at $O(\log n)$ instead of $O(n)$.

Figure 3: Lazy propagation, the idea behind three of the eight solutions.

A ninth structure lives outside that folder: a sparse table for static range minimums in `graphs/problem-59`, with $O(1)$ queries from two overlapping power-of-two blocks.

5 Verified, not assumed

- **Build check.** `tools/build_all.py` compiles every file in parallel with the flags a judge uses, `-std=gnu++17` and `-O2`. On 26 October 2026, in a clean `gcc:13` container: 392 of 392 files compile (361 of Salman's, 30 from other sources, the template).
- **Stress test.** `stress/stress.py` generates small random inputs (with values up to 10^9 half the time, to catch overflow), runs the solution and a Python brute force, and compares answers token by token, with a fixed seed so failures reproduce. All eight range-query solutions matched on 1,000 cases each.
- **The check can fail.** A deliberately broken minimum query failed on the first case.
- **Regression tests.** `stress/regressions.py` checks each of the 18 solutions fixed after review against a brute force written from its problem statement (or a checker, or a full game search), built with sanitizers, plus the template's repaired helpers. All pass; the old versions of the 12 real bugs fail the same tests.
- **One-off sweeps during curation** (not scripts in the repository): every file prints an answer, and every file waits for its input instead of crashing at startup (one USACO solution reads a named file instead, as USACO requires). Files that failed these were removed.

6 Code that is not Salman's

A file-by-file authorship pass found 30 files that are not his: 20 reference solutions headed with Priyansh Agarwal's name (CSES DP, one downloaded folder), and 10 contest solutions in other people's templates or styles (galen_colin's and jiangly's templates, someone else's debug header and input paths, editorial-style code, chatbot-style comments). Salman edited many of them while learning, so they stay in `from-other-sources/` with a README naming each one, and are excluded from every count. The first published version counted them; it was corrected within a day.

7 Honest limits

What a reader should know

- **What the checks cover.** Correct answers are machine-checked for the eight range-query solutions and the 18 fixed ones; for the other 335, the evidence is "it compiles and starts".
- **Bugs that were there.** Reading every file found 12 real bugs (out-of-bounds accesses, an operator-precedence slip, an uninitialised counter, a bit test that ignored its input, a wrong sort direction, and three solutions whose algorithm was wrong) and 6 latent ones. All are fixed and tested; the Deep Dive lists each one.
- **Template weight.** The full template's helpers were repaired (a discrete log that was off by one, CRT and discrete log that assumed a prime modulus, declarations that matched nothing), but it still allocates 41 MB of tables on every run (7 MB for the trimmed version).
- **Simpler would do.** Range add with point reads uses a lazy segment tree where a Fenwick tree on the difference array is a third of the code; the two-tag tree uses -1 as a sentinel that only works under CSES's limits.
- **Mess left in.** Commented-out earlier attempts, unused helpers, a few files with numbers but no problem names. None of it prints on a judge.

8 The story

1. About eight months of practice from CS 334 onward (Salman's figure; file dates cover April to November 2025, after a bulk copy reset many of them).
2. 25 October 2026: curated from 480 candidate files into a repository; duplicates, placeholders, backups and non-compiling attempts left out, everything compiled, the range queries stress-tested, an assistant-written ICPC 2026 folder excluded.
3. 26 October 2026: a second, file-by-file pass corrected the template history, moved 30 files by other people into their own folder, and dropped seven non-solutions. The count settled at 361.
4. Later the same day: every catalogue line was checked against its file, and the bugs that review found were fixed, each with a brute-force test.

If starting over: write the brute force first, keep helpers in one header, prefer the simplest structure that fits, and use version control from day one.

9 Glossary

Brute force the slow, obviously correct way to compute an answer; the reference a stress test compares against.

CSES a fixed set of 300 training problems by topic.

Fenwick tree an array of partial sums indexed by binary structure; point add and prefix sum in $O(\log n)$.

Lazy propagation deferring a range update by leaving a tag on a covering node.

$O(\log n)$ grows very slowly with input size: about 18 steps for 200,000.

Segment tree a binary tree of range summaries; any range is a handful of nodes.

Sparse table precomputed minimums of power-of-two blocks; $O(1)$ static queries.

Stress test many random small inputs, fast answer against brute force.

Template shared helper code pasted at the top of every solution.

Upsolving solving a problem after the contest, often from the editorial or others' code.